Steven Ma

Austin, TX · steven_c_ma@brown.edu
linkedin.com/in/steven--ma · github.com/stevenmaswim · stevenmaswim.github.io

Education

Brown University Providence, RI
Sc.B. (Bachelor of Science), Applied Mathematics–Computer Science · GPA: 4.0/4.0 *Expected May 2029*

- **Coursework:** Data Structures & Algorithms, Computing Foundations, Linear Algebra, Multivariable Calculus
  Current (Fall 2026): Fundamentals of Computer Systems, Probability & Statistics with Theory, Operations Research

Experience

Founder & Executive Director May 2024 – Present
KSJ Swimming *Austin, TX*

- Founded and scaled a swim-instruction business to $40K+ revenue and 300+ students across 100+ active clients, managing an instructor team and owning customer acquisition, scheduling, and finances, with SQL revenue reporting by period, coach, and pool
- Designed and shipped the production booking web app the business runs on, replacing paper tracking (ksjswimming.com · github); built access control into Postgres itself (Row-Level Security on **9** tables, a 3-tier role model, a Google-OAuth staff allowlist) and blocked double-booking with a partial unique index
- Shipped features from customer and staff feedback: unblocked families who could not book by raising a daily login-code cap from 40 to 200, added per-coach cancellation alerts, and built rain-out handling (dry run, undo) that keeps closures out of revenue
- Let one family book two kids at once by keying overlap checks per student and making multi-slot booking atomic on id-ordered row locks (70+ live-database tests); pre-launch audit found 0 Critical and fixed the one High (CSV formula injection in staff exports)
- Ran the platform at $0/month in hosting (serverless Deno edge function, Resend email, hourly pg_cron reminders); pinned every render and write to one business timezone with DST-correct conversion so customer emails never list a 4 PM lesson as 2 PM

Software Engineering Intern – ML & Data Jun 2026 – Aug 2026
Texas Department of Transportation (TxDOT) *Austin, TX*

- Built an end-to-end ML pipeline over **1.7M** bridge-inspection records (∼85K structures) from an entity-attribute-value schema via multi-join SQL; filtered out decommissioned bridges on supervisor feedback, and moved output into Snowflake for team SQL access
- Forecast bridge condition within ±1 point for **90–96%** of 68,959 held-out forecasts at 10 and 20 years; at 20 years, **1.5–1.6**× lower error than carry-forward and 12–21% below a Markov baseline; reported its one loss (10-year culverts)
- Caught **12** of 28 model inputs silently wired to NULL placeholders (each scored exactly 0.0 importance across all 4 models); reconciled 65 export headers to the schema, surfacing load-rating factors as top-5 drivers for every target in the first model
- Generated **575K** inspection-pair training examples by recasting the task as forecasting how each bridge deteriorates over time (time gap as an input), split by bridge ID (GroupShuffleSplit, test-enforced) so no bridge is in both train and test

Projects

Market-Data Ingestion Platform (prediction markets) · *Python, SQLite, pytest, systemd, Linux VPS* github

- Designed and ran a 5-service data platform on a Linux VPS that wrote 11.7M rows to SQLite: a recorder discovering markets every 5 min and polling order books every 15 s in live windows, a live-score detector, close and drift trackers, and a monitor
- Traced an API quota spike (∼500K tokens/day) to a change that kept re-fetching the vendor's full catalog; fixed it with a memoized catalog and a 1-hour circuit breaker on rate-limit errors

Order Matching Engine · *Python, pytest, threading, heapq, GitHub Actions* live demo · github

- Built a price-time-priority matching engine sustaining ∼**380K** orders/sec (5-run median on 12,000 seeded orders, committed in-repo), measuring 3-thread and single-threaded runs side by side to confirm the coarse lock is not the bottleneck
- Designed the book's data structures so cancel is $O(1)$: lazy deletion over `(price, seq, id)` heaps (flag and depth decrement, stale entries popped on read), with an arrival counter breaking ties FIFO and no custom comparator
- Made id assignment and heap insertion one atomic step so concurrent orders keep FIFO priority; after every stress test, invariant checks assert filled + remaining + cancelled = quantity, an uncrossed book, and maker-price trades (**56** unit tests, CI)

Systems Programming in C: Snake (CSCI 0300, Brown) · *C, Make, AddressSanitizer, Linux* Fall 2026

- In the course's starter C codebase, implemented UTF-8 code-point counting and rejected malformed run-length-encoded boards with distinct error codes; measured a 280-byte cap cutting 53 of 217 characters from a Hebrew message

Technical Skills

Languages: Python, Java, SQL, C, C++17 (learning), JavaScript, HTML/CSS

Frameworks & Data: PostgreSQL (Supabase, RLS), SQLite, Snowflake, pytest, Deno, scikit-learn, XGBoost, pandas, NumPy

Tools & Concepts: AI-Assisted Development (Claude Code), Data Structures & Algorithms, Concurrency, Transactions & Row Locking, REST APIs, Testing & CI (GitHub Actions), ETL / Data Pipelines, Scheduled Automation (pg_cron), Linux, Git

Activities & Leadership

NCAA Division I Varsity Swimming (Ivy League, ∼20 h/week) · USA Swimming Scholastic All-American · Swim Across America
Brown Derivatives Investment Group (Associate) · Bruno Gives Back